

ELEC5305 Project Proposal

1. Project Title

Environmental Sound Classification Using MFCC, Mel Spectrograms, and CNNs

2. Student Information

Full Name:

Student ID (SID):

GitHub Username:

GitHub Project Link:

3. Project Overview

Following Craig Jin's feedback, the research focus has been refined from a two-system MFCC–MLP versus log-Mel–CNN comparison to the effect of temporal preservation and acoustic representation. The revised question is: "How much environmental-sound classification performance is gained by preserving temporal structure rather than summarising features over a recording, and how does this depend on the representation?" Three systems are planned: (A) aggregated MFCC (mean and standard deviation) with an MLP, (B) time-resolved MFCC with a CNN, and (C) log-Mel spectrograms with the same CNN architecture as B. This better separates the time-aggregation question from the MFCC versus log-Mel comparison. A versus B remains a comparison of full systems because the classifier changes.

The evaluation plan was corrected to use all ten predefined UrbanSound8K folds. In each run, one entire fold is reserved for testing, another of the nine remaining folds for validation, and eight for training. Related excerpts retain their assigned folds; training-only statistics will be used for standardisation. Per-recording peak normalisation has been omitted to avoid intentionally removing recording-level amplitude variation.

Following the feedback, extensive augmentation and SNR robustness tests have been deferred. The priority is first to study prior UrbanSound8K literature and replication code, reproduce or adapt an appropriate baseline where practical, evaluate the three controlled systems, and analyse characteristic confusions such as drilling versus jackhammer. The implementation described below is original preliminary code; replication of Salamon and Bello's published baseline has not yet been established.

4. Results Presentation (Preliminary Implementation)

4.1 Development Environment and Signal Processing

The project is developed in VS Code on Windows using Python 3.12.10, librosa, NumPy, pandas, SciPy, scikit-learn, Matplotlib and PyTorch. CUDA acceleration was verified on an NVIDIA GeForce RTX 3070 Laptop GPU. Dependencies were captured in requirements.txt, with pip check reporting no broken requirements. A common config.json records the selected processing and training settings.

Parameter	Setting
Sampling rate	22,050 Hz
Clip duration	4 s (pad/truncate)
STFT window / FFT / hop	Hann / 1,024 / 512 samples
MFCC / Mel channels	40 / 128
Peak normalisation	Not applied

A synthetic four-second signal combined a 440 Hz tone, an increasing-frequency chirp and repeated 1,800 Hz bursts. The feature extractor produced STFT (513×173), MFCC (40×173), log-Mel (128×173), and aggregated MFCC (80,) arrays, as expected.

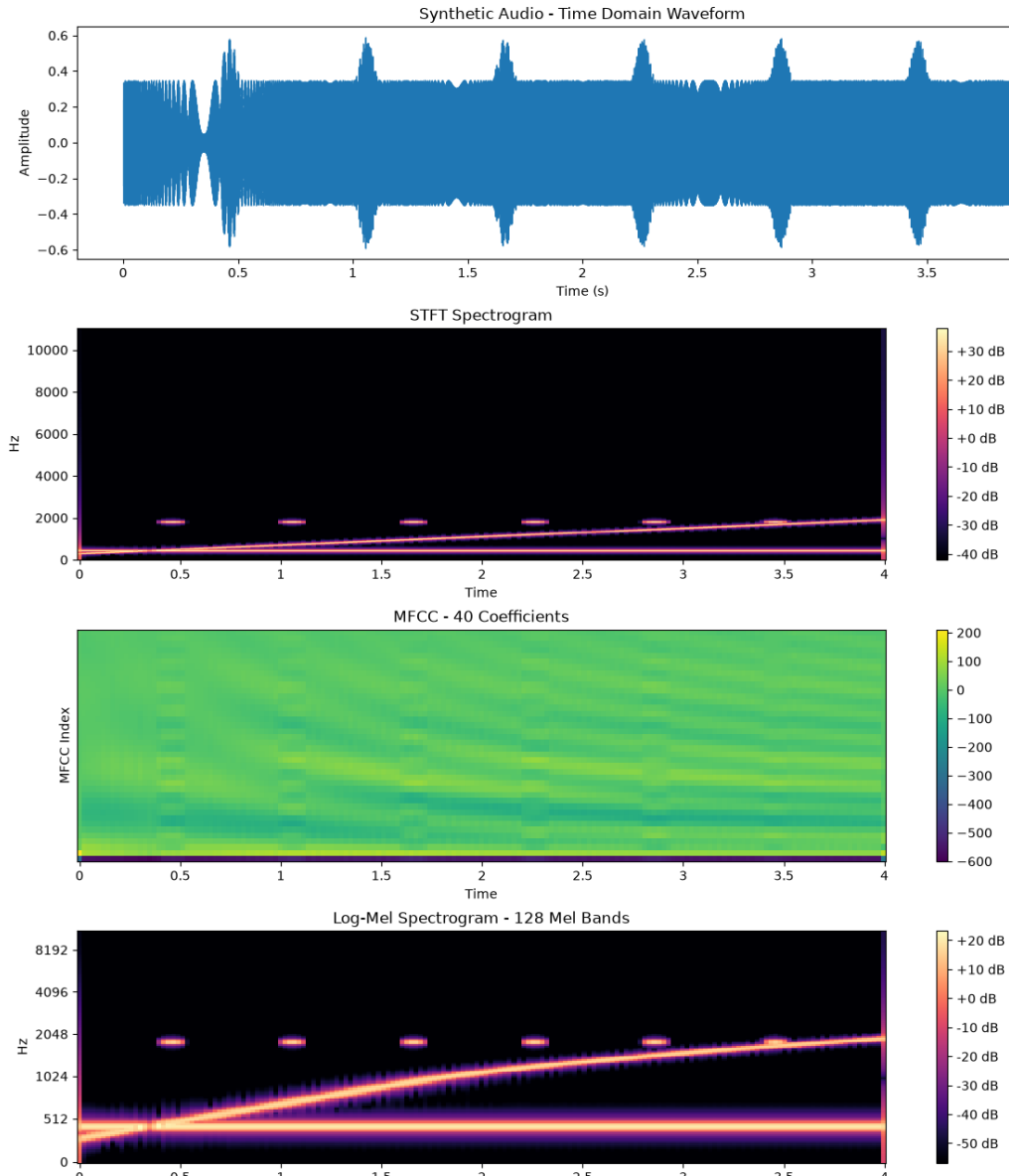


Figure 1. Synthetic waveform, STFT, time-resolved MFCC and log-Mel representations (software verification).

4.2 Temporal Structure and MFCC Aggregation

The 173 MFCC frames were randomly permuted using the same permutation for all 40

coefficients. This destroyed the original ordering without changing the frames themselves. Mean and standard deviation were then recalculated for each coefficient. Although the original and permuted heatmaps differed visually, their 80-dimensional summary vectors nearly coincided; small discrepancies arose from floating-point arithmetic. Thus temporal mean/std aggregation does not retain the sequence of acoustic events. This demonstrates an information-loss mechanism, not a classification-performance advantage for a CNN.

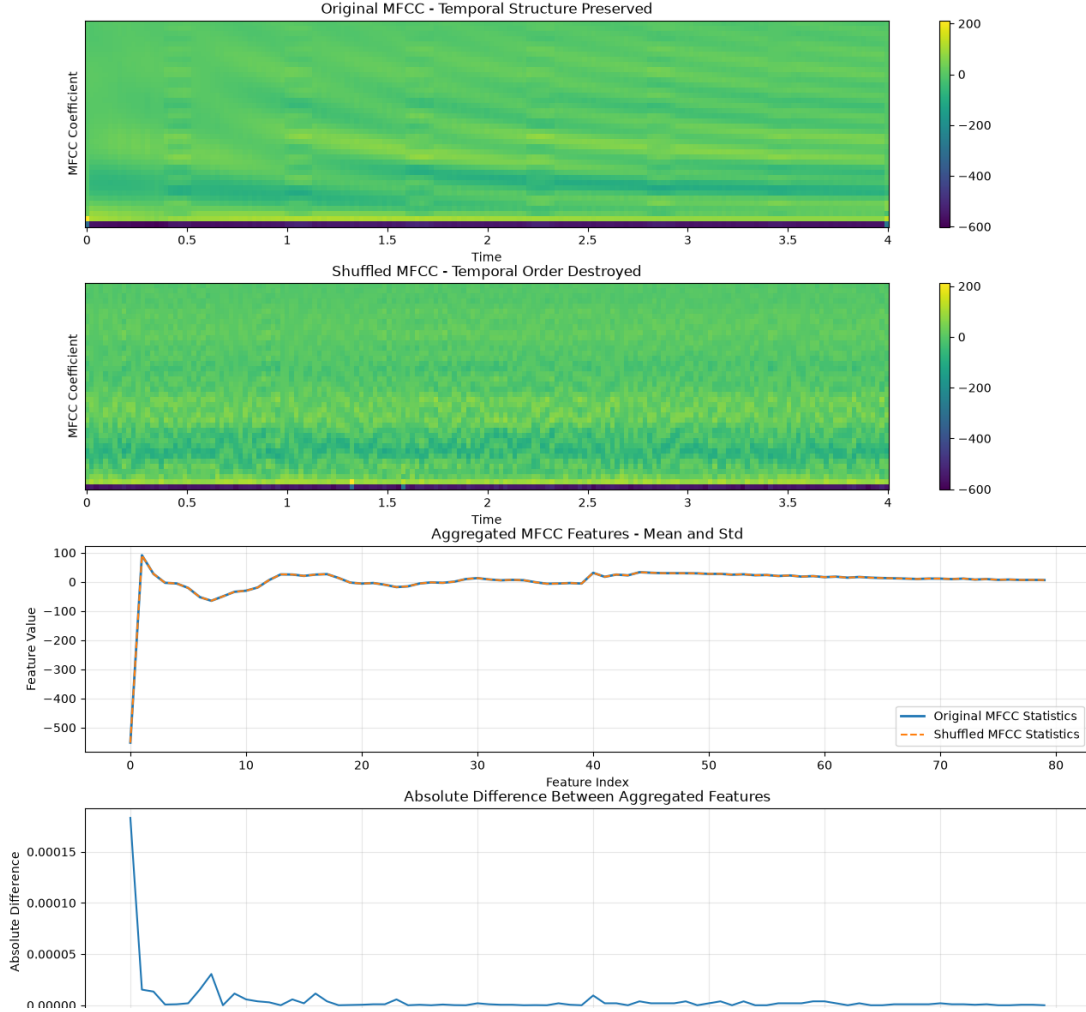


Figure 2. Original and shuffled MFCC frames, aggregated statistics and their absolute difference.

4.3 Model Implementation and Synthetic Functional Testing

Model A uses an $80 \rightarrow 128 \rightarrow 64 \rightarrow 10$ MLP with ReLU and 19,274 trainable parameters. Models B and C share the same 2D CNN: three convolutional blocks (16, 32, 64 channels), batch normalisation, ReLU, pooling, adaptive average pooling, dropout and a ten-class head. Each CNN has 24,170 trainable parameters. Their respective inputs are 40×173 MFCC and 128×173 log-Mel arrays.

The train/validation/test routines, backward propagation, checkpoints, fold-labelled feature caches and metric calculations were exercised with synthetic signals, feature maps or artificial labels. A shared training configuration specifies batch size 32, Adam learning rate 0.001, maximum 50 epochs and patience 8. CNN functionality tests deliberately ran only five epochs. The MLP synthetic test exercised early stopping.

Functional test	Accuracy	Macro-F1	Important limitation
MLP (synthetic features)	100.0%	1.0000	Artificial separable class centres
MFCC-CNN (synthetic maps)	77.5%	0.7190	Artificial feature patterns
Log-Mel CNN (synthetic maps)	60.0%	0.4833	Different artificial feature dimensions/patterns

These figures are software self-test outputs only; they are NOT UrbanSound8K results and do not establish which acoustic representation performs better. The MLP learning curves confirm that optimisation and result saving execute as intended.

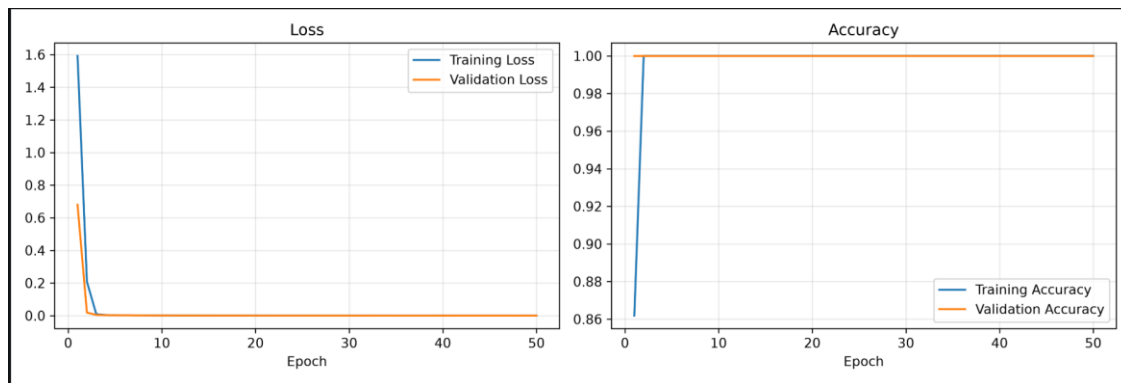


Figure 3. Synthetic MLP training and validation curves; not an environmental-sound benchmark.

4.4 Dataset Pipeline and Ten-Fold Evaluation

The data-processing modules can save labels and original fold IDs alongside aggregated MFCC, time-resolved MFCC and log-Mel features. Miniature synthetic WAV datasets verified extraction, cache save/load and fold preservation. The evaluate.py self-test verified ten distinct test-fold assignments, 30 experiment commands (three models $\times$ ten folds), and mean/standard-deviation summary generation using explicitly artificial fixture values.

The complete UrbanSound8K archive was still downloading when this progress report was prepared. Consequently, no extraction of all 8,732 real clips, published-baseline reproduction, actual ten-fold classification results, or real per-class confusion analysis is claimed.

5. Code & Results

Module	Function / verified stage
audio_features.py; visualize.py	Synthetic waveform/STFT/MFCC/log-Mel and temporal shuffle visualisations
dataset.py; cnn_dataset.py	Feature caching, labels and fold metadata; synthetic self-tests
train_mlp.py; train_cnn.py	Synthetic training, validation, evaluation and checkpoint saving

evaluate.py	Ten-fold scheduling and summary self-test
model_metrics.py	Accuracy, macro-F1, per-class recall, confusion matrix, latency utility
config.py; config.json	Central configuration loader and experiment settings

The repository named in Section 2 is the same one supplied in the proposal. Local code and figures should be synchronised to the repository before submission; inclusion of its URL here does not verify that the latest files have been pushed.

6. Discussion and Next Steps

The preliminary evidence supports correct software operation and the expected loss of temporal ordering after MFCC mean/std aggregation. It does not yet quantify improvement in environmental-sound recognition. The main methodological limitation is the absence of real UrbanSound8K test outcomes.

1. Complete the official UrbanSound8K download; inspect filenames, metadata, 8,732 entries and all ten official folds.
2. Inspect relevant prior work and available replication code; document which published baseline is genuinely reproduced or adapted.
3. Create the three feature caches and verify identical filenames, labels and folds across representations.
4. Run an initial real-data fold, then all ten test folds for each model; fit normalisation only on training folds.
5. Report mean $\pm$ standard deviation for accuracy and macro-F1, class-wise recall, confusion matrices, parameter counts and controlled inference latency.
6. Investigate confused categories through waveforms, MFCC trajectories and log-Mel patterns; add only a small motivated augmentation check if time permits.

7. References

- [1] J. Salamon, C. Jacoby, and J. P. Bello, “A dataset and taxonomy for urban sound research,” *Proc. ACM Multimedia*, pp. 1041–1044, 2014.
- [2] K. J. Piczak, “Environmental sound classification with convolutional neural networks,” *Proc. IEEE MLSP*, 2015.
- [3] S. B. Davis and P. Mermelstein, “Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences,” *IEEE Trans. ASSP*, 28(4), 357–366, 1980.
- [4] J. Salamon and J. P. Bello, “Deep convolutional neural networks and data augmentation for environmental sound classification,” *IEEE Signal Processing Letters*, 24(3), 279–283, 2017.
- [5] K. Choi, G. Fazekas, M. Sandler, and K. Cho, “A comparison of audio signal preprocessing methods for deep neural networks on music tagging,” *Proc. EUSIPCO*, 2018.
- [6] B. McFee et al., “librosa: Audio and music signal analysis in Python,” *Proc. SciPy*, 2015.
- [7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [8] UrbanSound8K: <https://urbansounddataset.weebly.com/urbansound8k.html>
- [9] Salamon2017Replication: <https://github.com/grudloff/Salamon2017Replication>
- [10] UrbanSound8K-JAMS: <https://github.com/justinsalamon/UrbanSound8K-JAMS>